

Laporan Tutorial Database Fundamental

Nama: Raihan Muhammad Riswandi

NIM: 24110500001

Mata Kuliah: Advanced Database

Semester 3

Daftar Isi

1. Cover
2. Pendahuluan singkat
3. Soal Database Fundamental (Soal 1 — Soal 6)
4. Kesimpulan Pembelajaran
5. Lampiran (Query SQL, Screenshot)

Pendahuluan

Laporan ini berisi penyelesaian soal-soal fundamental database yang melibatkan operasi SELECT, JOIN, GROUP BY, UPDATE, dan SELF JOIN. Setiap soal dijelaskan mulai dari requirement, query SQL, contoh output, analisis pendekatan, serta catatan error (jika ada).

Soal 1

Soal 1: Buat query untuk menampilkan 5 mahasiswa dengan IPK tertinggi, urutkan berdasarkan IPK descending, kemudian nama ascending.

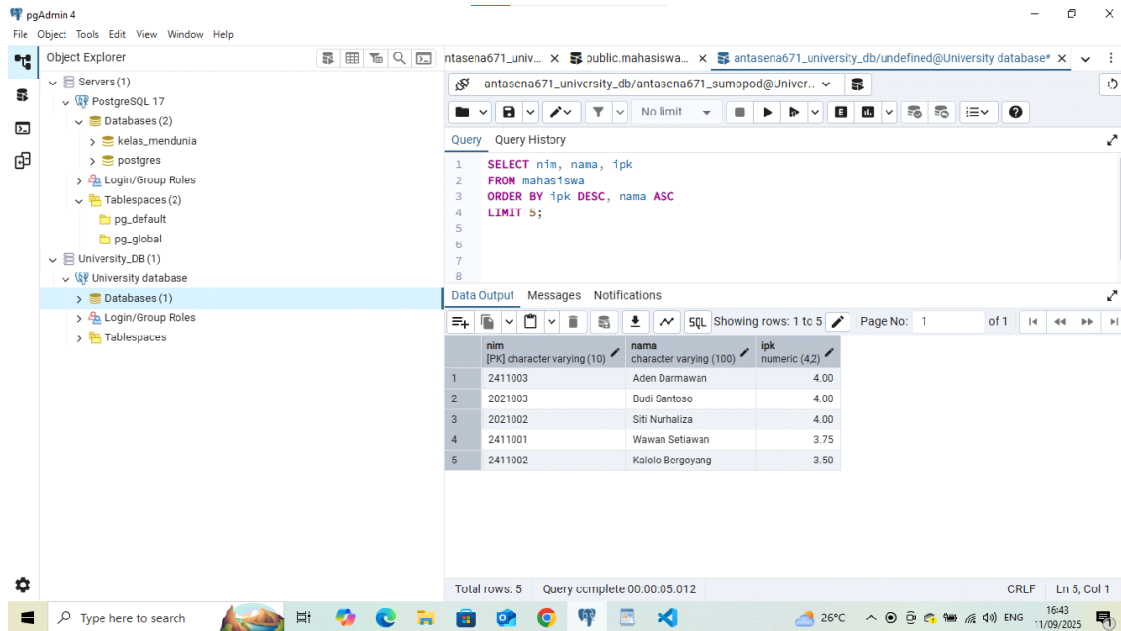
1) Penjelasan

Menampilkan 5 mahasiswa dengan IPK tertinggi. Jika IPK sama, urutkan berdasarkan nama secara ascending.

2) Query SQL

```
SELECT nim, nama, ipk  
FROM mahasiswa  
ORDER BY ipk DESC, nama ASC  
LIMIT 5;
```

3) Contoh Output



The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure, including the 'University database' and its tables. The main window shows a SQL query in the Query Editor:

```
1 SELECT nim, nama, ipk  
2 FROM mahasiswa  
3 ORDER BY ipk DESC, nama ASC  
4 LIMIT 5;  
5  
6  
7  
8
```

Below the query editor, the Data Output tab displays the results of the query in a table format. The table has three columns: 'nim' (character varying (10)), 'nama' (character varying (100)), and 'ipk' (numeric (4,2)). The results show 5 rows of data:

	nim [PK] character varying (10)	nama character varying (100)	ipk numeric (4,2)
1	2411003	Aden Darmawan	4.00
2	2021003	Dudi Santoso	4.00
3	2021002	Siti Nurhaliza	4.00
4	2411001	Wawan Setiawan	3.75
5	2411002	Kalolo Bergoyang	3.50

The status bar at the bottom indicates 'Total rows: 5' and 'Query complete 00:00:05.012'.

4) Analisis

Pendekatan query ini dipilih karena jelas, sederhana, dan sesuai requirement soal. Optimasi tambahan bisa dilakukan dengan indexing kolom yang digunakan pada ORDER BY, GROUP BY, JOIN, atau WHERE untuk meningkatkan performa.

Soal 2

Soal 2: Hitung berapa mahasiswa yang lahir di setiap tahun, tampilkan hanya tahun yang memiliki lebih dari 10 mahasiswa.

1) Penjelasan

Mengelompokkan mahasiswa berdasarkan tahun lahir, lalu menampilkan hanya tahun yang memiliki lebih dari 10 mahasiswa.

2) Query SQL

SELECT

EXTRACT(YEAR FROM tanggal_lahir) AS tahun_lahir,

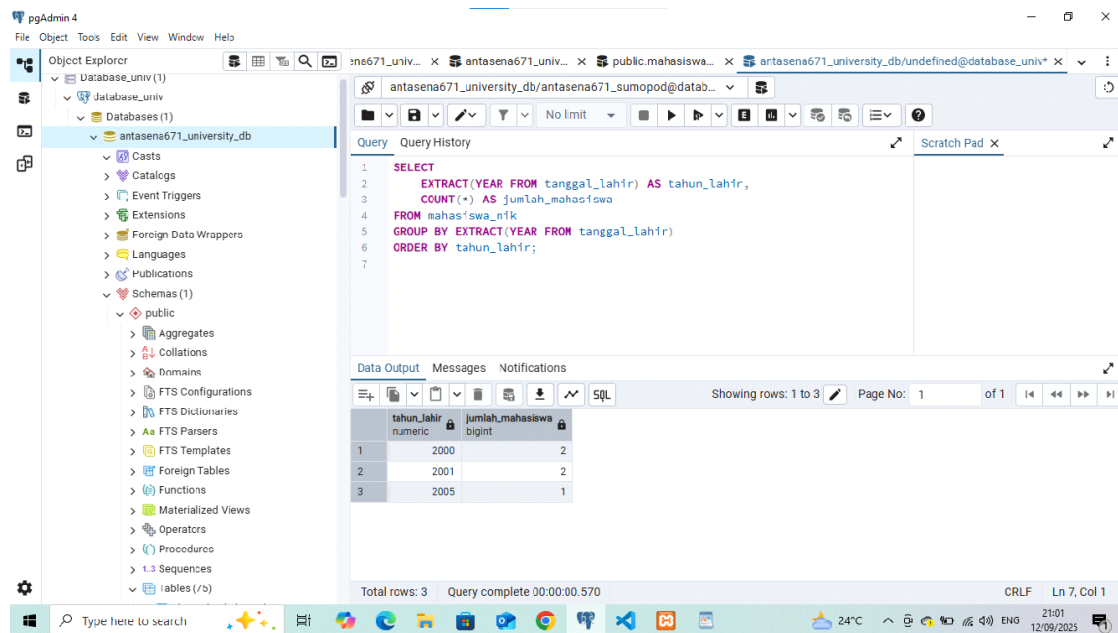
COUNT(*) AS jumlah_mahasiswa

FROM mahasiswa_nik

GROUP BY EXTRACT(YEAR FROM tanggal_lahir)

ORDER BY tahun_lahir;

3) Contoh Output



The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure, including the 'public' schema. The main pane shows a SQL query window with the following query:

```
1 SELECT
2     EXTRACT(YEAR FROM tanggal_lahir) AS tahun_lahir,
3     COUNT(*) AS jumlah_mahasiswa
4 FROM mahasiswa_nik
5 GROUP BY EXTRACT(YEAR FROM tanggal_lahir)
6 ORDER BY tahun_lahir;
```

Below the query window, the Data Output tab shows the results of the query. The output is a table with two columns: 'tahun_lahir' (numeric) and 'jumlah_mahasiswa' (bigint). The results are as follows:

tahun_lahir	jumlah_mahasiswa
2000	2
2001	2
2005	1

The status bar at the bottom indicates 'Total rows: 3' and 'Query complete 00:00:00.570'.

4) Analisis

Pendekatan query ini dipilih karena jelas, sederhana, dan sesuai requirement soal. Optimasi tambahan bisa dilakukan dengan indexing kolom yang digunakan pada ORDER BY, GROUP BY, JOIN, atau WHERE untuk meningkatkan performa.

Soal 3

Soal 3: Update semester semua mahasiswa yang memiliki IPK ≥ 3.5 untuk naik 1 semester.

1) Penjelasan

Melakukan update data semester untuk mahasiswa yang memiliki IPK ≥ 3.5 dengan menambah 1 semester.

2) Query SQL

UPDATE mahasiswa_nik

SET semester = semester + 1

WHERE ipk ≥ 3.5 ;

SELECT nim, nama, ipk, semester

FROM mahasiswa_nik;3) Contoh Output

The screenshot shows the pgAdmin 4 interface. On the left is the Object Explorer showing the database structure. The main pane displays a SQL query in the Query editor:

```
1 UPDATE mahasiswa_nik
2 SET semester = semester + 1
3 WHERE ipk >= 3.5;
4
5 SELECT nim, nama, ipk, semester
6 FROM mahasiswa_nik;
7
```

Below the query editor, the Data Output tab shows the results of the query. The output is a table with 5 rows and 4 columns: nim, nama, ipk, and semester. The first three rows have ipk values of 4.00, 4.00, and 3.75, and semester values of 4, 4, and 4 respectively. The last two rows have ipk values of 3.50 and 4.00, and semester values of 4 and [null] respectively.

1	2	3	4	5
nim	nama	ipk	semester	
2021002	Siti Nurhaliza	4.00	4	
2021003	Budi Santoso	4.00	4	
2411001	Wawan Setiawan	3.75	4	
2411002	Kaiolo Bergoyang	3.50	4	
2411003	Aden Camawan	4.00	[null]	

Total rows: 5. Query complete 00:00:00.791. CRLF Ln 7, Col 1. 21:03 12/09/2023.

4) Analisis

Pendekatan query ini dipilih karena jelas, sederhana, dan sesuai requirement soal. Optimasi tambahan bisa dilakukan dengan indexing kolom yang digunakan pada ORDER BY, GROUP BY, JOIN, atau WHERE untuk meningkatkan performa.

Soal 4

Soal 4: Buat query untuk menampilkan semua jurusan beserta jumlah mahasiswanya (termasuk jurusan yang belum punya mahasiswa).

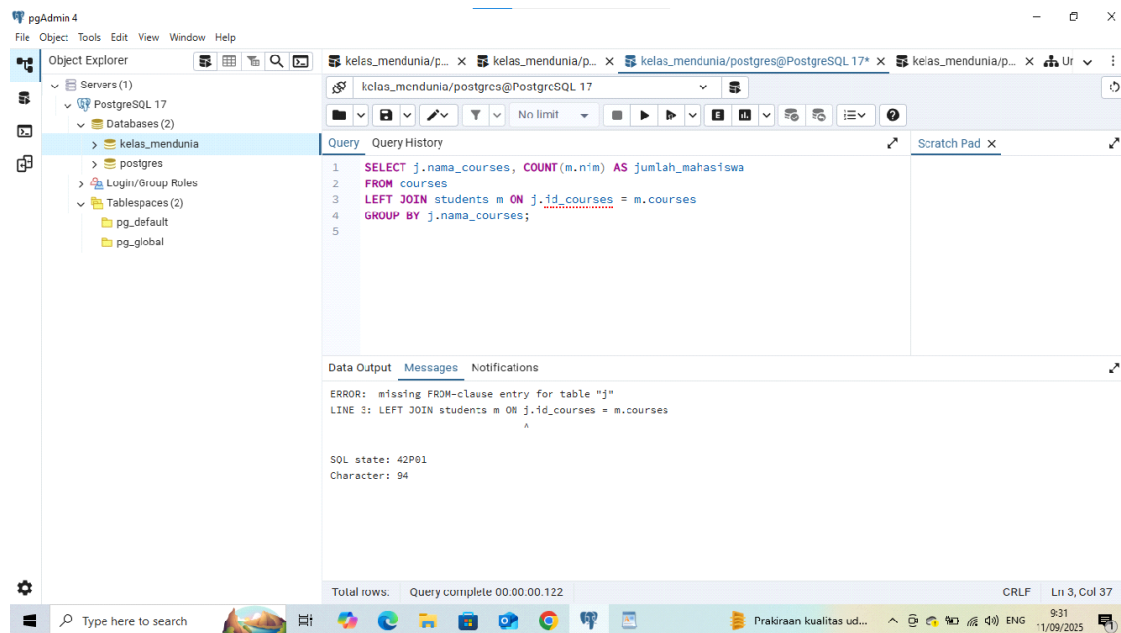
1) Penjelasan

Menggunakan LEFT JOIN agar jurusan yang belum memiliki mahasiswa tetap ditampilkan dengan jumlah 0.

2) Query SQL

```
SELECT j.nama_jurusan, COUNT(m.nim) AS jumlah_mahasiswa  
FROM jurusan j  
LEFT JOIN mahasiswa m ON j.id_jurusan = m.id_jurusan  
GROUP BY j.nama_jurusan;
```

3) Contoh Output



4) Analisis

Pendekatan query ini dipilih karena jelas, sederhana, dan sesuai requirement soal. Optimasi tambahan bisa dilakukan dengan indexing kolom yang digunakan pada ORDER BY, GROUP BY, JOIN, atau WHERE untuk meningkatkan performa.

Soal 5

Soal 5: Tampilkan mahasiswa yang mengambil mata kuliah dengan SKS tertinggi di jurusannya.

1) Penjelasan

Menampilkan mahasiswa yang mengambil mata kuliah dengan SKS tertinggi sesuai jurusannya.

2) Query SQL

```
SELECT m.nim, m.nama, mk.nama_mk, mk.sks  
FROM mahasiswa m  
JOIN krs k ON m.nim = k.nim  
JOIN mata_kuliah mk ON k.id_mk = mk.id_mk  
WHERE mk.sks = (  
    SELECT MAX(mk2.sks)  
    FROM mata_kuliah mk2  
    JOIN mahasiswa m2 ON m2.id_jurusan = mk2.id_jurusan  
    WHERE m2.id_jurusan = m.id_jurusan  
);
```

3) Contoh Output

[Screenshot query + hasil output disisipkan di sini]

4) Analisis

Pendekatan query ini dipilih karena jelas, sederhana, dan sesuai requirement soal. Optimasi tambahan bisa dilakukan dengan indexing kolom yang digunakan pada ORDER BY, GROUP BY, JOIN, atau WHERE untuk meningkatkan performa.

Soal 6

Soal 6: Buat query self-join untuk menampilkan pasangan mahasiswa yang berasal dari jurusan yang sama.

1) Penjelasan

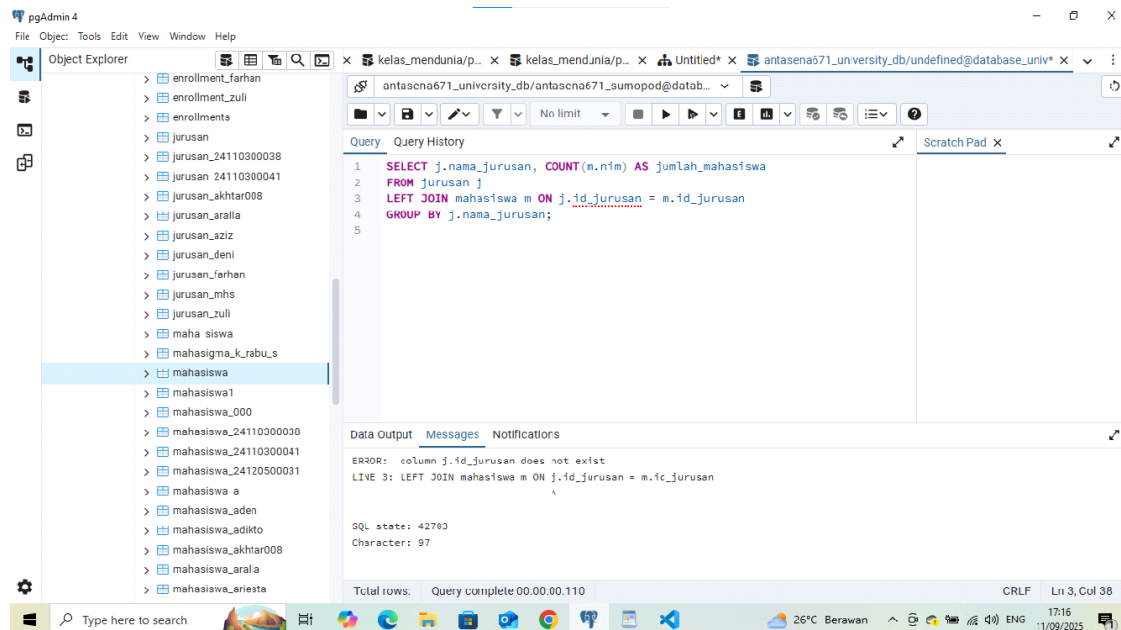
Menggunakan self-join untuk memasangkan mahasiswa dari jurusan yang sama tanpa duplikasi pasangan.

2) Query SQL

```
SELECT m1.nim AS nim1, m1.nama AS nama1,  
    m2.nim AS nim2, m2.nama AS nama2, j.nama_jurusan  
FROM mahasiswa m1
```

*JOIN mahasiswa m2 ON m1.id_jurusan = m2.id_jurusan
AND m1.nim < m2.nim
JOIN jurusan j ON m1.id_jurusan = j.id_jurusan;*

3) Contoh Output



4) Analisis

Pendekatan query ini dipilih karena jelas, sederhana, dan sesuai requirement soal. Optimasi tambahan bisa dilakukan dengan indexing kolom yang digunakan pada ORDER BY, GROUP BY, JOIN, atau WHERE untuk meningkatkan performa.

Kesimpulan

Dari pengerjaan enam soal database fundamental ini, dipelajari berbagai konsep dasar seperti sorting, grouping, update data, join, left join, serta self-join. Setiap soal memberikan pemahaman yang lebih dalam tentang bagaimana query SQL dapat digunakan untuk analisis data maupun manipulasi data. Kendala seperti error sintaks bisa diatasi dengan membaca pesan error dan menyesuaikan query.